

LLM-Based Translation Pipeline for Functional Coverage Analysis

Pavel Lukl*

Abstract

PLACEHOLDER Safety-critical avionics software under DO-178C requires formal coverage analysis of requirements, but requirements are written in natural language. Manual formalization is slow, error-prone, and does not scale. This report investigates the use of large language models (LLMs) in a multi-step agentic pipeline to automatically translate natural-language requirements into a formal predicate-based representation. We survey existing approaches to requirement formalization, evaluate four LLM orchestration frameworks, and prototype a pipeline with ambiguity review, entity extraction, formula generation, and structural validation. We implemented and compared PydanticAI, Haystack, LangGraph, and CrewAI on a set of real avionics requirements using a locally-run 14B parameter model. CrewAI produced the deepest formulas and fastest execution, while PydanticAI offered the least code complexity and best schema evolution support. The model capability, not the framework, was the primary bottleneck for complex requirements. The pipeline architecture is viable for automating requirement formalization. This work establishes the foundation for a production system and identifies concrete next steps: deeper semantic validation, verification via roundtrip checking, tool integration, and evaluation with stronger models.

Keywords: DO-178C — LLM — requirement formalization — agentic pipeline — functional coverage

Supplementary Material: N/A

*xluklp00@stud.fit.vut.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

Safety-critical avionics software must demonstrate that tests adequately cover the behaviors specified in requirements. This coverage analysis requires a formal representation of the requirements, yet in practice, both requirements and test cases are written in natural language. Without formalization, inconsistencies between requirements and tests can go undetected, potentially compromising the safety of the final system. Translating them into a formal form is therefore essential, but it is currently a manual process: slow, expensive, and prone to human error. As the number of requirements grows, manual formalization becomes a bottleneck in the certification workflow.

The broader project addresses this by building an

end-to-end system: natural language requirements and test cases are translated into a formal representation, the translations undergo thorough validation, and finally a gap analysis identifies missing coverage between the formalized requirements and test cases. This report focuses on the first stage: automating the translation using large language models (LLMs). Other parts of the project (the formal representation itself, the semantic validation of translations, and the gap analysis) are being developed in parallel by other team members.

Rather than relying on a single LLM prompt, we design an agentic pipeline of several steps, where each step produces inspectable, validated output, a property essential in safety contexts where traceability of failures is required. The pipeline takes a natural language

requirement as input and produces a formal predicate tree conforming to a formalism developed within the project.

We begin by surveying existing approaches to requirement formalization and the technologies available for building LLM pipelines in this context (Section 2). Drawing inspiration from these, we draft our own approach to the translation problem, describe the pipeline architecture and the design decisions behind it, and present initial experiments comparing four orchestration frameworks on real avionics requirements (Section 3). Based on the findings, we outline concrete directions for future development as the project continues (Section 4).

2. State of the Art

Translating natural language requirements into formal representations has traditionally been a manual process requiring domain expertise. Recent advances in large language models have opened the possibility of automating this translation, and a growing body of work explores different strategies, from fine-tuning and prompt engineering to constrained decoding and pipelines composed of multiple LLM calls. We first survey these approaches and the methodological patterns they share (Section 2.1), then briefly review the concrete tooling available for building such pipelines (Section 2.2).

2.1 LLM-Based Requirement Formalization

A recent systematic literature review by Beg et al. [1] identifies 35 studies on using large language models to formalize natural language requirements, published between 2022 and mid-2025. The review organizes the literature along several dimensions (target formalism, prompting strategy, degree of automation, and evaluation methodology) and classifies prompting strategies into zero-shot, few-shot, chain-of-thought, iterative/human-in-the-loop, and fine-tuning categories. We structure the following discussion around the main technical approaches that have emerged, progressing from fine-tuning to interactive methods, constrained decoding, and pipelines with multiple stages.

Fine-tuning. Hahn et al. [4] show that pre-trained language models can be fine-tuned on parallel corpora of natural language sentences and their formal equivalents, achieving competitive translation accuracy. A key finding for our purposes is that fine-tuned models generalize to unseen variable names, a property that rule-based and grammar-based approaches lack. Their experiments target first-order logic (FOL), linear-time temporal logic (LTL), and regular expressions. The

main limitation is the data requirement: fine-tuning needs a sufficiently large dataset of natural language sentences paired with their correct formal translations, which is expensive to construct for specialized domains such as avionics where no such dataset exists.

Interactive decomposition and dynamic prompting.

Two techniques that avoid the data cost of fine-tuning are interactive sub-translation and dynamic few-shot prompting (where a small number of solved examples are included in the prompt to guide the model). Cosler et al. [2] propose decomposing the formalization into sub-translations: the LLM maps fragments of the input to sub-formulas, which serve as a structured intermediate representation that users can inspect, edit, and correct before the final formula is assembled. This decomposition makes the model’s reasoning transparent at each step, and the interactive loop lets users resolve ambiguities that single-pass approaches cannot reliably detect. The framework, called *nl2spec*, targets LTL and similar temporal logics. Xu et al. [6] take the prompting strategy further with dynamic prompt generation: instead of using a fixed set of few-shot examples, the system retrieves the most relevant examples from a prompt library via embedding similarity for each input, and evolves the library over time by incorporating corrected translations. This achieves up to 94.4% accuracy without human interaction. With interactive prompt evolution, accuracy on an out-of-domain dataset improves from 27% to 78%. Their target formalism is also LTL. Both approaches demonstrate that careful prompt selection and human-in-the-loop refinement can compensate for a model’s limited domain knowledge without requiring fine-tuning, and that inspectable intermediate outputs are valuable for catching errors early.

Constrained decoding. Rather than validating or retrying after generation, Grammar-Forced Translation (GraFT) by English et al. [3] constrains the language model’s token generation at each step so that only tokens permitted by the target grammar can be produced. This provides a formal guarantee that every generated formula is syntactically valid, eliminating the need for structural retries entirely. The technique involves two steps transferable to other formalisms: first, a masked language model lifts atomic propositions from the natural language input (replacing domain terms with integer placeholders), then a fine-tuned sequence-to-sequence model generates the formula under grammar constraints. In their evaluation on temporal logic benchmarks, this improves end-to-end accuracy by 5.49% and out-of-distribution accuracy by 14.06%

over prior methods.

Multi-LLM and agentic pipelines. Several recent works split the formalization task into a sequence of LLM calls, each responsible for one part of the problem and producing its own structured output. Yan et al. [7] demonstrate a three-phase architecture: the first LLM converts unstructured text into a structured description using a unified template, the second generates formal output from these descriptions, and the third validates the results. The template intermediate representation between phases ensures consistency and enables validation before proceeding to the next stage. Although their application domain is hardware verification assertions, the architectural pattern of extracting structured descriptions via templates, then generating and validating formal output, is directly applicable. Nouri et al. [5] apply a similar decomposition strategy in the automotive safety domain: they break the analysis into smaller tasks with dedicated prompts for each, where each step’s output feeds the next without modification. Their key finding is that this decomposition with a dedicated prompt for each step significantly outperforms a single monolithic prompt, an observation confirmed across multiple expert evaluation cycles. Together, these works establish that task decomposition, structured intermediate representations between stages, and validation at each stage are effective and transferable strategies for complex formalization tasks.

2.2 Technologies

Building this kind of pipeline requires concrete choices about how the steps are orchestrated and how the LLM is accessed. Several open-source frameworks have emerged for this purpose. PydanticAI provides the most direct integration with Pydantic output schemas, with automatic retry on validation failure built in. Haystack models pipelines as directed acyclic graphs of typed components, with native support for serialization to YAML. LangGraph represents pipelines as state graphs with conditional edges and supports cycles, which naturally accommodates loops that retry on validation failure. CrewAI takes a role-based approach, defining agents with goals and backstories that collaborate on tasks. Other notable frameworks include DSPy, which optimizes prompts programmatically, and Instructor, which wraps function-calling APIs with Pydantic validation. The choice of framework determines how easily the pipeline can be inspected at intermediate steps, how schema changes propagate, and how validation errors are surfaced, all of which directly affect auditability. The LLM itself can be accessed either through cloud APIs (such as Claude or GPT-4) or by

running open-weight models locally using tools like Ollama. Most orchestration frameworks support both options, allowing the same pipeline to switch between them with minimal code changes.

Summary

The surveyed literature shows that LLMs can translate natural language requirements into formal representations with increasing accuracy, and that the most effective approaches share several methodological patterns: task decomposition into smaller, individually verifiable steps; structured intermediate representations between stages; and feedback-driven refinement, whether through human interaction, dynamic prompt evolution, or automated retry on validation failure. On the correctness-guarantee spectrum, strategies range from post-hoc validation with retry [2, 6], through extraction guided by templates [7], to compile-time guarantees via grammar-forced decoding [3]. The existing work targets either generic temporal logics (LTL, STL) or assertion languages specific to hardware verification and automotive safety. The formalism targeted in our project, a predicate representation developed specifically for avionics that can be mapped to LTL and similar temporal logics, makes the LTL-focused literature directly informative, but the specific combination of avionics domain, predicate output formalism, and a pipeline architecture with validation at each stage remains unaddressed. Our work targets this gap.

3. Our Approach

3.1 Goal and Research Questions

3.2 Inspiration and Design Decisions

3.3 Pipeline Architecture

3.4 Experiments

4. Future Work

5. Conclusion

References

- [1] BEG, A., GERVASI, V., VIDANAGE, K. and LIYANAGE, L. H. A Systematic Literature Review on Formalising Natural Language Requirements Using Large Language Models. *ArXiv preprint*. 2025. arXiv:2506.11874v1.
- [2] COSLER, M., HAHN, C., MENDOZA, D., SCHMITT, F. and TRIPPEL, C. nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models. In: *Proceedings of the 35th International Conference on Computer Aided Verification (CAV)*. 2023.

- [3] ENGLISH, W., SIMON, D., JHA, S. K. and EWETZ, R. Grammar-Forced Translation of Natural Language to Temporal Logic using LLMs. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. 2025.
- [4] HAHN, C., SCHMITT, F., TILLMAN, J. J., METZGER, N., SIBER, J. et al. Formal Specifications from Natural Language. *ArXiv preprint*. 2022. arXiv:2206.01962v1.
- [5] NOURI, A., CABRERO DANIEL, B., TÖRNER, F., SIVENCORONA, H. and BERGER, C. Engineering Safety Requirements for Autonomous Driving with Large Language Models. In: *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE)*. 2024.
- [6] XU, Y., FENG, J. and MIAO, W. Learning from Failures: Translation of Natural Language Requirements into Linear Temporal Logic with Large Language Models. In: *Proceedings of the IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. 2024.
- [7] YAN, Z., FANG, W., LI, M., LI, M., LIU, S. et al. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In: *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC)*. 2025.